第 7 次课逐字讲稿 v4（课堂现场精修版）

课程：软件体系结构｜授课教师：何庭钦｜对象：2025 级电子信息工程硕士

版本说明：本版保留 PPT 01-45 的页码对应关系，在 v3 基础上强化课堂现场话术、互动操作、学生可能回答与教师点评；弱化说课感，适合打印后现场授课参考。

课堂节奏建议

- 90 分钟版：PPT 01-06 导入 12 分钟；PPT 07-19 可测试性与测试层级 35 分钟；PPT 20-28 质量门禁与契约测试 20 分钟；PPT 29-39 安全/缺陷/AI 15 分钟；PPT 40-45 演练与收束 8 分钟，课堂演练可转为课后。
- 120 分钟版：保留 PPT 40-42 现场演练 25-30 分钟，并在 PPT 19、24、30、35 增加学生回答与点评。

【PPT 01】测试策略与质量保障体系

讲授方式：正常讲｜建议时长：2 分钟

本页目标：让学生从一开始就知道，本讲不是测试工具课，而是从架构视角讨论质量如何被验证。

现场逐字稿

我们今天进入“测试策略与质量保障体系”。我先把边界说清楚：这不是一节 JUnit、pytest 或 Selenium 的工具课，也不是把单元测试、集成测试、端到端测试背一遍。今天真正要回答的是一个更工程化的问题：你画出来的架构，怎样证明它真的可靠？

前面几讲，我们已经能把系统组织起来：预约服务、审批服务、通知服务、数据库、权限层、部署单元，都能画在图上。但真实项目里，图画完以后马上会有人追问：并发预约真的不会重复占用吗？普通学生真的进不了管理员接口吗？通知服务挂掉时主流程会不会被拖垮？

生活里也一样。一辆车不能只靠宣传说“很安全”，它要做碰撞测试、刹车测试、极限路况测试。软件质量也不是上线前让测试同学“测一测”才出现的，它必须在架构边界、依赖关系、可观测性和质量门禁里提前设计好。

所以今天这讲的主线只有一句话：质量不是最后测出来的，而是被架构设计出来、被测试验证出来、被门禁守住的。

互动/板书提示

开场设问：如果一个系统功能都能跑，但没人能证明它在并发和越权场景下正确，你会认为它完成了吗？停 5 秒，让学生先形成问题感，不必马上点名。

转场句：接下来先把本讲放回整门课的链条里，看它承接上一讲和下一讲的位置。

【PPT 02】从架构设计到质量验证

讲授方式：正常讲/快速带过｜建议时长：2 分钟

本页目标：说明本讲承接分布式架构设计，转入质量验证，不是另起一门测试课。

现场逐字稿

上一讲我们讲的是系统怎样组织：服务怎么拆，数据怎么流，接口怎么连，部署单元怎么安排。这些内容解决的是“结构设计”的问题。今天往后追一步：这些结构决策能不能被验证。 比如上一讲画出预约服务调用通知服务，这条线不是画出来就结束。我们要问：通知服务不可用时，预约服务是 否还能完成？这条边是否可以在测试环境里替换？接口错误时能不能定位？再比如数据库一致性，如果两个学生 同时预约同一设备，系统是靠业务代码判断，还是靠数据库锁和事务保证？这些都不是单纯测试人员能事后补救 的问题。 下一讲我们会进入发布、交付和运维。一个系统只有通过了质量验证，才谈得上进入发布流程。否则发布只是把 不确定性推给用户。

互动/板书提示

请学生回想上一讲本组画过的架构图，快速想一个“最难验证的箭头”或“最难验证的组件”。可 以让 1 名学生说出例子，教师点评时强调：难验证的地方通常就是架构风险点。

转场句：明确课程链条以后，我们看本讲结束时各组需要形成什么产出。

【PPT 03】M4 产出：测试计划 V1 与质量门禁清单

讲授方式：正常讲/快速带过｜建议时长：3 分钟

本页目标：把测试计划和质量门禁与 M4 里程碑绑定，明确不是额外作业，而是架构验证成果。

现场逐字稿

这页是本讲的成果要求。我们最后不是要求大家交一堆测试用例，而是交两类东西：测试计划 V1 和质量门禁清单。 测试计划 V1 要回答三个问题：第一，哪些风险需要测；第二，用哪一层测试去测；第三，什么结果算通过。比如 “通知服务降级”这个风险，不能只写“测试通知服务”，而要写清楚：停止通知服务或用 Mock Server 模拟失败，调用预约创建接口，期望预约记录仍创建成功，不返回 500，并且通知失败进入重试队列。 质量门禁清单则更像工程里的安检口。机场安检不是看起来差不多就让你过，而是一条条检查。软件门禁也一样：单元测试是否 100% 通过，核心集成测试是否通过，P0 缺陷是否为 0，越权访问是否返回 403。它必须能被检查，最好能被 CI/CD 自动阻断。

互动/板书提示

点名提醒：M4 不看文档是否写得长，而看标准是否可检查。可让学生判断：“系统性能较好”能否 作为门禁？预期答案：不能，因为不可量化。

转场句：有了产出目标，我们回到最尖锐的入口问题：你的架构到底能不能被验证。

【PPT 04】导入问题：你的架构能被验证吗？

讲授方式：互动停留/详细讲｜建议时长：4 分钟

本页目标：用三个具体问题让学生意识到，测试困难的根本往往是架构边界不清。

现场逐字稿

我们先不讲概念，先看三个非常像真实项目的问题。

第一个，预约冲突出了 Bug。两个学生几乎同时预约同一台设备，系统最后出现了两条有效预约。这个时候，你能不能在 5 分钟内判断：问题到底在业务规则、数据库事务、锁机制，还是接口调用？

第二个，通知服务挂了。学生预约成功以后，本来应该收到短信或邮件。现在通知服务不可用，预约流程到底应该失败，还是预约成功、通知稍后重试？这个答案不是测试人员临时决定的，而是架构设计时就要定义好的降级策略。

第三个，普通学生拿着自己的 Token 访问管理员接口。系统能不能稳定返回 403？还是某个 Controller 忘了写权限判断，就直接放过去了？

大家注意，这三个问题表面上都是“测试问题”，但根子都在架构上。能不能定位、能不能隔离、能不能替换依赖、能不能观察状态，都是架构提前决定的。

互动/板书提示

现场操作：问“这三个问题更像测试问题，还是架构问题？”给 10 秒举手。若学生说测试问题，回应：对，表面是测试问题，但测试能否写出来取决于结构。若学生说架构问题，回应：对，尤其是权限层、依赖隔离和事务边界。

转场句：接下来，我们把第 4 讲的质量属性场景拿回来，看看一个质量目标怎样变成测试方案。

【PPT 05】从质量属性场景反向追问测试方案

讲授方式：正常讲｜建议时长：3 分钟

本页目标：让学生理解质量属性场景必须继续落到测试层级、环境和通过标准。

现场逐字稿

第 4 讲我们写过质量属性场景，比如“100 个学生同时提交预约，P95 响应时间不超过 2 秒，重复占用数为 0”。这 已经比“系统性能要好”清楚很多，因为它有刺激源、有环境、有响应、有指标。 但从工程角度看，它还差一步：怎样证明它满足？如果你只是写了 P95 小于 2 秒，但不知道用 JMeter 还是单元测试，不知道需要真实数据库还是 Mock，不知道失败以后看日志还是看数据库记录，那这个指标仍然停留在纸面。 可以把它类比成考试复习。你说“我要数学考 90 分以上”，这只是目标；你还要说每天做几套题，错题怎么复盘，模拟考试低于多少分就补哪一章。软件质量目标也要有这样的验证动作。

互动/板书提示

让学生口头补全三个问题：在哪一层测？需要什么环境？失败后如何定位？如果学生回答“压力测 试”，继续追问：压哪里？走浏览器还是直接压 API？数据库是真实还是内存？

转场句：这就引出今天的核心判断：质量不是最后测出来的，而是被设计出来的。

【PPT 06】核心判断：质量是被设计出来的

讲授方式：正常讲｜建议时长：3 分钟

本页目标：确立全讲主线：测试发现问题，架构决定发现和定位问题的成本。

现场逐字稿

这页是今天最重要的一句话：质量是被设计出来的。 测试当然重要，它能暴露缺陷。但测试本身不能自动降低定位成本。比如同样是通知失败，如果通知服务是通过 接口注入的，测试时可以用 Mock 替换，失败就能定位到预约服务的处理逻辑；如果代码里直接硬编码 HTTP 地址 ，测试时就必须启动真实通知服务，失败可能来自网络、服务、配置、测试数据，定位成本一下子上去了。 所以我们今天讲的“质量保障”，不是把责任推给测试人员，而是让架构设计本身支持验证。清晰边界帮助定位 ，可替换依赖帮助隔离，可观测状态帮助判断，可量化门禁帮助团队停止冒险。

互动/板书提示

可以问：一个测试失败后，如果你只能说“系统整体不对”，这说明测试写得不好，还是架构粒度 太粗？点评时强调二者有关，但架构粒度是根。

转场句：先从第一个概念开始：什么叫可测试性。

【PPT 07】可测试性：能被高效、精准地验证

讲授方式：正常讲｜建议时长：3 分钟

本页目标：准确解释可测试性，并用生活化类比说明它衡量的是诊断和验证成本。

现场逐字稿

可测试性，英文叫 Testability，指系统在多大程度上能被高效、精准地验证。 这里有两个关键词。高效，是测试运行快、环境少、反馈及时。比如你改了预约时长校验，应该几秒钟就知道是 否正确，而不是必须启动前端、后端、数据库、邮件服务。精准，是失败后能定位到具体函数、模块、接口或外 部依赖，而不是只看到“提交预约失败”。 生活里可以用设备维修来理解。一个设备如果预留了检修口，哪个模块坏了可以快速拆开检查；如果所有零件焊 死在一起，哪怕只是一个小传感器坏了，也要拆整机。软件可测试性就是有没有这些“检修口”。接口、依赖注 入、日志、事件、状态返回，都是软件里的检修口。

互动/板书提示

提示学生把可测试性记成两个词：高效和精准。可以让学生复述：高效对应什么？精准对应什么？

转场句：那么架构里哪些条件会直接影响可测试性？看下一页四个条件。

【PPT 08】可测试性的四个架构条件

讲授方式：正常讲｜建议时长：4 分钟

本页目标：讲清可测试性的四个架构条件，并说明每个条件为什么会影响测试。

现场逐字稿

判断一个系统是否容易测试，可以看四个架构条件。 第一，边界清晰。业务逻辑、数据访问、外部调用不要混在一起。如果预约时长校验写在 Controller 里，审批状态 又写在 SQL 里，前端还偷偷做一部分判断，测试时就不知道该测哪一层。 第二，依赖可替换。数据库、通知服务、第三方认证服务，在测试环境里要能换成 Mock、Stub 或测试容器。就像 排练节目，不一定每次都把灯光、音响、舞台全部搭起来，很多时候可以用替身先排动作。 第三，状态可观测。测试完以后要能看见结果：返回值、数据库记录、日志、事件、审计记录。第四，输入输出 可控。时间、随机数、并发条件不能完全不可控。比如审批超时 48 小时，如果代码直接取系统当前时间，测试就 很难稳定复现边界。

互动/板书提示

快速提问：如果一个功能依赖当前时间 now()，哪个条件受影响？预期答案：输入输出可控。补充点 评：可以注入 Clock 或 TimeProvider。

转场句：接下来用一段代码看这些条件怎样具体影响测试成本。

【PPT 09】反面代码：业务逻辑与外部依赖耦合

讲授方式：互动停留/详细讲｜建议时长：5 分钟

本页目标：通过反面代码逐行说明耦合写法如何把单元测试推向集成测试。

现场逐字稿

我们看这段反面代码。先说明，它不是语法错误，也不是只有初学者才会写。很多真实项目早期为了快，都会把代码写成这样。

第一个问题是 `db.execute`。预约服务内部直接访问数据库。于是我们只想测“设备不可用时不能预约”这个规则，也必须准备数据库表、插入设备记录、清理测试数据。一个本来应该很轻的业务规则测试，立刻变重。

第二个问题是 `requests.post`。业务逻辑里直接调用通知服务。通知服务慢、挂了、地址配错了，都会让预约测试失败。也就是说，测试预约逻辑时，通知服务变成了必需品。

所以这段代码的最大问题不是功能一定错，而是验证成本高。测试失败以后，你不知道失败来自业务规则、数据库连接、网络、通知服务，还是测试数据。课堂上我们要看的是这个：结构一旦耦合，测试就会被迫升级，定位也会变粗。

互动/板书提示

发问：如果现在要测“设备已被占用时不发送通知”，这段代码需要准备什么？给 10 秒。预期答案：数据库、通知服务、测试数据。点评：这就是单元测试被迫变成小型集成测试。

转场句：那怎样把这段代码变得容易测？看依赖注入写法。

【PPT 10】正面代码：依赖注入让外部依赖可替换

讲授方式：互动停留/详细讲｜建议时长：5 分钟

本页目标：讲清依赖注入不是框架概念，而是把外部依赖变成测试时可替换对象。

现场逐字稿

再看正面写法。不要把依赖注入理解成 Spring 的形式主义，它的本质很简单：外部依赖不要在业务代码里写死，而是从外面传进来。

这里 ReservationService 不再自己决定怎么查数据库、怎么发通知，而是依赖 `DeviceRepository` 和 `NotificationClient` 两个边界。测试时，我们可以传一个假的 DeviceRepository，让它固定返回“设备已占用”；再传一个假的 NotificationClient，用来检查有没有发送通知。

这样一来，测试不需要真实数据库，也不需要真实通知服务。失败时也更容易判断：外部依赖都被我们控制住了，问题大概率就在 ReservationService 的业务逻辑里。

可以类比练车。耦合写法像每次练刹车都必须直接上高速；依赖注入像先在训练场练刹车、转向、倒车。训练场不等于真实道路，但它能让你低成本验证关键动作。单元测试也是这个逻辑。

互动/板书提示

追问：依赖注入之后，失败更可能定位到哪里？预期答案：ReservationService 的业务逻辑。点评：因为外部依赖已经被替身控制住了。

转场句：把两种写法放到一起看，测试成本差异就非常明显。

【PPT 11】同一功能，两种架构的测试成本差异

讲授方式：正常讲｜建议时长：3 分钟

本页目标：用工程后果解释为什么架构选择会影响 CI 速度、定位精度和团队信任。

现场逐字稿

同一功能，两种架构，最后不只是代码风格不同，而是工程成本不同。 耦合写法下，测试需要数据库、通知服务、完整数据准备，跑得慢，而且失败定位粗。依赖注入下，外部依赖可 以替换，测试毫秒级运行，失败基本指向业务逻辑。 这里要强调一个企业实践里的后果：如果 CI 每次跑测试要 40 分钟，团队慢慢就不愿意频繁提交；如果测试经常因 为环境问题失败，团队就会开始怀疑测试结果。最后出现一种很危险的状态：测试红了，大家第一反应不是“代 码

有问题”，而是“是不是测试又抽风”。一旦到这一步，自动化测试就失去门禁价值。 可以把这件事放到学生项目里看：一开始大家为了赶进度，把数据库查询、权限判断、通知发送都写在一个函数 里，功能确实很快能跑。但到了第 4 周需要改规则、补测试、做演示时，成本会集中爆发。好的架构不是让第一天 写得更快，而是让后面每一次修改都能被低成本验证。

互动/板书提示

让学生判断：CI 慢一定是测试太多吗？预期答案：不一定。点评：也可能是架构没有隔离依赖，导 致大量测试都变成慢测试。

转场句：接下来总结一下哪些架构特征通常会导致可测试性差。

【PPT 12】可测试性差的五种典型架构特征

讲授方式：正常讲/快速带过｜建议时长：3 分钟

本页目标：帮助学生识别自己项目中可测试性差的常见结构信号。

现场逐字稿

这五类特征，是学生项目和企业遗留系统里都很常见的可测试性风险。 第一，静态方法或全局工具类直接访问数据库。看起来方便，但测试很难隔离。第二，HTTP 地址硬编码，服务之 间直接互相调用，测试就必须依赖完整服务网络。第三，业务逻辑混在 Controller 层。这样规则只能通过 HTTP 请 求触发，无法单独验证。第四，全局状态和单例让测试之间互相污染，前一个测试留下的数据会影响后一个测试。 第五，时间和随机数不可控，比如审批超时、验证码、抽签分配，如果不能控制输入，测试就不稳定。 大家可以回去检查自己的项目：哪些地方改一个小规则必须启动整个系统？哪些地方测试失败以后不知道原因？ 这些地方就是架构债务的信号。

互动/板书提示

让学生在本组代码中想一个例子：有没有 Controller 里写了大量业务判断？有没有硬编码外部 URL？ 不要求公开回答，作为课后检查点。

转场句：有了可测试性视角，我们再看测试金字塔。

【PPT 13】测试金字塔：不是背图，而是资源配置

讲授方式：互动停留/详细讲｜建议时长：4 分钟

本页目标：纠正把测试金字塔当比例图背诵的误解，强调它是反馈成本模型。

现场逐字稿

测试金字塔大家可能见过很多次。底下是大量单元测试，中间是适量集成测试，顶上是少量端到端测试。但它真正要表达的不是一个固定比例，不是 70%、20%、10% 这种机械数字，而是一个反馈成本模型。

越靠底层，测试越快、越便宜、越容易定位，所以可以多写。越靠顶层，越接近真实用户，但也越慢、越脆、越难定位，所以要少而精。

用考试复习来理解最直观。平时做选择题、计算题，相当于单元测试，成本低、反馈快；阶段测验相当于集成测试，看知识点之间能不能接起来；期末模拟考相当于端到端测试，最接近真实考试，但不可能每天都来一套三小时大考。

软件测试也是一样：不能用最高成本的方法验证所有细节。越是小规则，越应该在低层快速验证；越是完整信心，才交给少量端到端测试。

互动/板书提示

问：如果预约时长边界值也用浏览器端到端测试来测，会有什么问题？预期答案：慢、定位困难、成本高。点评：真实不等于高级，适合才是关键。

转场句：下一页把金字塔和架构层对应起来，这是考试和项目都要掌握的表。

【PPT 14】测试层级 × 架构层对应关系

讲授方式：互动停留/详细讲｜建议时长：5 分钟

本页目标：让学生掌握测试层级选择的核心规则：风险在哪个架构层，就优先在哪层验证。

现场逐字稿

这页是本讲最核心的知识表之一。请大家不要只记“单元、集成、端到端”三个词，要记它们对应的架构层。 单元测试验证函数、类和领域模型逻辑。它的速度是毫秒级，成本极低，定位可以精确到函数。比如预约时长 1 到 4 小时、学生是否具备预约资格、状态能否从待审批转到已取消，这些都适合单元测试。 集成测试验证模块协作和接口契约，对应服务边界、数据访问层、消息队列、认证链。比如审批通过以后是否触 发通知，预约成功以后设备状态是否更新，50 个并发请求下数据库是否只产生一条有效预约，这些都需要集成测 试。 端到端测试验证完整业务链路，对应整个系统和外部依赖。它最真实，但速度慢、成本高、定位难。判断规则很 简单：风险发生在哪一层，就优先在哪一层测。不要一遇到“真实场景”就上端到端。

互动/板书提示

让学生跟读式记忆三句话：单元测规则，集成测协作，端到端测链路。随后问：50 并发预约同一设 备是哪一层？预期答案：集成测试，因为需要真实数据库但不需要浏览器。

转场句：下面我们分别看三层测试在预约系统里具体测什么。

【PPT 15】单元测试：把业务规则单独验证清楚

讲授方式：正常讲｜建议时长：3 分钟

本页目标：具体说明单元测试适合验证业务规则、边界值和状态机。

现场逐字稿

单元测试这一层，最适合把业务规则单独验证清楚。 比如预约时长必须是 1 到 4 小时整数，这就是典型边界值：0 小时不行，1 小时可以，4 小时可以，5 小时不行，小 数不行。再比如学生是否完成培训、是否超出预约上限，预约状态能不能从待审批转到通过或拒绝，这些都是纯 逻辑规则。 单元测试不适合验证邮件是否真的发出、数据库是否真的写入、前端表单是否能提交。它的价值是让业务规则从 Controller、SQL 和页面脚本中抽出来，成为可以单独验证的对象。就像做数学题，先把每个公式练熟，不要每次 都做整张综合卷。 这也是为什么我们一直强调领域模型和应用服务层。只要规则被抽出来，单元测试就像给每条规则单独做小测验 ；如果规则散落在页面、Controller 和 SQL 里，就只能做大综合题，每次反馈都很慢。

互动/板书提示

小问答：验证“预约时长必须是 1-4 小时整数”用哪层？预期答案：单元测试。追问：为什么不需要 数据库？点评：因为它是纯规则。

转场句：当我们要验证两个模块接起来以后是否正确，就进入集成测试。

【PPT 16】集成测试：验证模块接起来以后是否正确

讲授方式：正常讲｜建议时长：4 分钟

本页目标：说明集成测试关注模块协作、真实数据库行为和关键边界。

现场逐字稿

集成测试不是“比单元测试大一点”这么简单，它关注的是边界协作。 预约系统里有很多事情必须接起来才知道对不对。预约服务创建记录以后，设备库存是否被标记为占用？审批通 过以后，通知服务是否收到消息？学生 Token 访问管理员接口，认证和授权过滤链是否真正生效？这些都不是一 个函数自己能证明的。 并发一致性尤其典型。50 个请求同时预约同一设备，如果只用 Mock Repository，它可能全部按你写的假逻辑运行 ，看不出数据库事务、唯一约束、乐观锁有没有问题。所以并发场景通常要用真实数据库或测试容器，用 API 压 测而不是浏览器点页面。

互动/板书提示

发问：并发预约为什么通常不是单元测试？等待 10 秒。可能答案：因为需要真实数据库。点评：对 ，关键风险在数据写入和事务边界，不在单个函数。

转场句：再往上，就是端到端测试，它覆盖完整用户路径，但数量必须克制。

【PPT 17】端到端测试：只保留最高价值的完整链路

讲授方式：正常讲｜建议时长：3 分钟

本页目标：说明端到端测试的价值和边界，避免学生把所有真实场景都放到 E2E。

现场逐字稿

端到端测试最接近真实用户。学生登录、查看设备、提交预约、教师审批、学生收到通知，这是一条完整业务链 路。这样的测试很有价值，因为它能证明主流程从前端到后端再到外部依赖整体打通。 但端到端测试也最贵。浏览器可能不稳定，测试数据可能脏，网络可能慢，外部服务可能临时不可用。它失败时 ，定位粒度也很粗：你只知道“提交预约失败”，还要继续查是前端表单、后端接口、数据库、权限还是通知服 务。 所以端到端测试应该覆盖最高价值的主流程和少量高风险路径，不要拿它测所有边界值。边界值交给单元测试， 模块协作交给集成测试，完整信心由端到端测试提供。

互动/板书提示

问：学生登录到提交预约完整流程适合哪层？预期答案：端到端。再问：预约时长 5 小时不允许，是 否需要端到端？预期答案：不需要，单元或接口层更合适。

转场句：如果把所有测试都压到端到端，会发生什么？看一个失败案例。

【PPT 18】失败案例：所有测试都压在端到端

讲授方式：互动停留/详细讲｜建议时长：5 分钟

本页目标：用失败案例讲出 E2E 过重的现象、后果、根因和启发。

现场逐字稿

这个案例很常见。一个团队认为“只有端到端测试才是真实测试”，于是大量使用 Selenium 或 Playwright，从登录页面一路点到提交结果。刚开始大家很有信心，觉得测试覆盖了真实用户路径。 几个月后问题出现了：CI 每次运行 35 分钟，偶尔因为浏览器、网络、测试数据失败。失败越来越多，大家开始说 “这次红灯可能又是环境问题”。再后来，看到红灯也不急着修，自动化测试就变成了摆设。 表面原因是 E2E 太多，深层原因是架构不可测。业务逻辑散落在 Controller 和页面里，底层没有清晰领域对象，依 赖也不可替换，所以团队写不了有效单元测试，只能从 UI 层触发。启发是：端到端过重常常不是测试策略单独的 问题，而是架构没有提供底层测试入口。

互动/板书提示

发问：这个案例的表面原因是什么？深层原因是什么？可能答案：表面是 E2E 太多，深层是业务逻 辑不可单测。点评：对，要从测试现象反推架构问题。

转场句：现在我们用四个小场景检查大家是否能选择正确测试层级。

【PPT 19】课堂判断：四个场景分别放在哪一层测？

讲授方式：互动停留/详细讲｜建议时长：5 分钟

本页目标：通过互动题训练学生按风险位置选择测试层级。

现场逐字稿

这一页我们不急着公布答案。大家先把 A、B、C、D 四个场景分别写成“单元、集成、端到端”中的一个。给大家 30 秒。

A，预约时长必须是 1 到 4 小时整数。它是纯业务规则，不依赖数据库，不依赖外部系统，单元测试最合适。

B，审批通过后通知服务能收到消息。这里要验证审批模块和通知模块协作，所以是集成测试，可以用内存 MQ 或 Mock Server 替代真实通知服务。

C，学生从页面提交预约到收到通知邮件。这是完整用户链路，适合端到端测试，但数量要少。

D，50 个并发预约同一设备，只有 1 人成功。这个最容易误判为端到端，因为它听起来很真实。但关键风险在数据库并发和事务控制，不需要浏览器，所以更适合 API 级集成测试。

这道题的收束规则只有一句：测试层级看风险位置，不看场景听起来是否真实。

互动/板书提示

操作：先让学生独立判断 30 秒，再重点问 D。若有人选端到端，追问“不经过浏览器，直接打 API 能不能验证关键风险？”引导到“真实数据库 + API 并发”即可。

转场句：测试层级选对以后，还需要把质量要求变成能阻断流程的门禁。

【PPT 20】质量门禁：进入下一阶段的可量化标准

讲授方式：互动停留/详细讲｜建议时长：4 分钟

本页目标：定义质量门禁，强调可量化、可检查、可阻断。

现场逐字稿

质量门禁，英文叫 Quality Gate。它是构建或版本进入下一阶段前必须满足的一组可量化标准。

没有门禁的项目里，最常见的一句话是：“这个版本基本没问题，先上吧。”危险就在“基本”两个字。什么叫基本？P0 缺陷有没有？核心链路通过率多少？越权测试跑了吗？性能 P95 是多少？这些说不清，质量就变成了感觉。

质量门禁要像安检口：规则明确，不满足就不能过。单元测试失败不能合并，关键集成测试失败不能进测试环境，P0 缺陷未关闭不能发布。

它还有一个很现实的价值：减少争论。没有门禁时，团队会讨论“这个问题严不严重”“能不能先上”。有门禁时，先看条件是否满足。不满足就修，满足才进入下一步。门禁把质量决策从情绪和压力里抽出来。

互动/板书提示

问：“测试基本通过”能不能作为门禁？预期答案：不能。点评：门禁必须能被不同人一致判断，最好能被机器自动检查。

转场句：实际项目里，门禁通常分三层设置。

【PPT 21】三层质量门禁：代码层、构建层、发布层

讲授方式：正常讲｜建议时长：4 分钟

本页目标：讲清代码层、构建层、发布层三类门禁的职责差异。

现场逐字稿

一套实用的质量门禁，通常分三层。 代码层门禁发生在每次提交或合并前，检查单元测试、静态分析、覆盖率、代码评审。它的目标是把低级错误挡 在最早阶段。就像作业提交前先检查格式和基本计算，不要等期末考试才发现公式写错。 构建层门禁发生在合并主分支或构建测试环境时，检查系统能否启动、核心集成测试是否通过、性能基线是否达标、安全依赖有没有高危漏洞。 发布层门禁发生在上线前，检查核心端到端场景、回归测试、负载测试、P0/P1 缺陷、回滚方案。越往后，问题越 贵，所以不是把检查都堆到发布前，而是每一层挡住适合它挡的问题。 三层门禁也对应三种反馈速度。代码层反馈最快，几分钟内告诉你基本规则有没有破坏；构建层反馈中等，告诉 你模块接起来是否正常；发布层反馈最重，告诉你系统是否具备交付条件。不要把轻量问题拖到重量级阶段。

互动/板书提示

让学生判断：预约冲突并发测试应该放在哪层门禁？可以接受答案：构建层或发布前必须项。点评 ： 关键是不要等最终演示前第一次跑。

转场句：下一页给一份预约系统 M4 可以直接参考的门禁示例。

【PPT 22】预约系统 M4 质量门禁示例

讲授方式：互动停留/详细讲｜建议时长：5 分钟

本页目标：示范如何把门禁写成可勾选、可评审的具体清单。

现场逐字稿

这页是预约系统的门禁示例。请注意这些表达的共同点：它们都可以检查。 “创建、审批、取消三条核心流程的集成测试全部通过”，评审时可以看测试报告。“50 并发同一设备，201 等于 1 条、409 等于 49 条、数据库记录等于 1 条”，这是精确的通过标准。“学生 Token 访问 /admin/** 返回 403”，这 是安全门禁。 推荐项也要有数值，比如预约服务域覆盖率达到 65%，设备状态机达到 80%。推荐不等于随便写，如果没达到，也 要在评审中说明原因。门禁写得越清楚，团队越少争论“能不能先过”。 这里的“必须”和“推荐”也要区分清楚。必须项是底线，不满足就不能过；推荐项是质量提升项，不满足可以 解释，但不能假装不存在。课程项目里不要求你们达到企业生产标准，但要求你们知道标准应该怎样写。

互动/板书提示

让学生把“并发测试通过”改写成可检查门禁。预期答案包含并发数、接口、响应数量、数据库记 录。点评：只有能被机器或评审者检查，才叫门禁。

转场句：这些门禁如果只写在文档里还不够，最好嵌进 CI/CD 流水线。

【PPT 23】CI/CD 中的门禁位置

讲授方式：正常讲/快速带过｜建议时长：3 分钟

本页目标：解释质量门禁嵌入 CI/CD 后如何成为工程事实。

现场逐字稿

这页看流水线位置。代码提交以后，先跑代码门禁：单元测试、静态分析、覆盖率。通过以后才能合并。合并后 进入构建门禁：BVT、集成测试、性能基线。通过以后才进入测试环境。发布前再跑发布门禁：端到端、回归、负 载、缺陷清零和回滚方案。 门禁只有嵌入流水线，才不是口头约定。否则项目压力一大，就很容易出现“这次情况特殊，先跳过”。但越是 紧急发布，越需要门禁。因为人在紧急状态下最容易漏掉风险，机器化门禁反而能保持底线。 如果你们以后进入企业研发，会发现门禁常常写在 GitLab CI、GitHub Actions、Jenkins 或流水线平台里。它不是一 个老师检查用的表格，而是合并按钮、发布按钮前面的自动检查。架构课程讲门禁，就是要让设计决策能进入工 程流程。

互动/板书提示

判断题：紧急修复能不能跳过质量门禁？可能答案：可以跳过部分低风险门禁。教师点评：可以有 分级策略，但关键安全、数据一致性、回滚门禁不能跳过；跳过必须记录风险和责任。

转场句：那怎样判断一个门禁写得好不好？看坏门禁和好门禁的对比。

【PPT 24】坏门禁与好门禁的差别

讲授方式：互动停留/详细讲｜建议时长：4 分钟

本页目标：训练学生把模糊质量要求改写为可量化门禁。

现场逐字稿

坏门禁往往听起来正确，但无法执行。“性能可接受”“基本没有严重缺陷”“测试基本通过”，这些都不能作 为门禁。因为不同人理解不同，项目压力一大就会被解释掉。 好门禁要包含对象、数值、条件和后果。例如“核心接口 P95 小于 500ms”，“P0 缺陷等于 0”，“核心集成测试 通过率等于 100%”。如果不满足，流水线停止或评审不通过。 判断规则可以记四个词：量化、复现、阻断、责任。能量化，说明结果不是感觉；能复现，说明别人能重复验证；能阻断，说明它有流程力量；责任清楚，说明失败后知道谁处理。 课堂上经常看到这样的写法：“系统稳定运行”“接口性能较好”“安全性较高”。这些话作为愿望可以，作为 门禁不行。软件工程强调可验证，就是要把愿望翻译成别人能复现的检查条件。

互动/板书提示

现场改写：“系统响应比较快”怎么改？可能答案：P95≤2s、错误率≤0.1%、并发 100 用户。点评： 越具体越能成为工程约束。

转场句：讲完门禁，我们看一个现代分布式系统中常见但容易误用的测试方法：契约测试。

【PPT 25】契约测试前先问：有几个独立部署服务？

讲授方式：互动停留/详细讲｜建议时长：4 分钟

本页目标：在讲契约测试前先建立适用条件判断，避免学生盲目引入复杂技术。

现场逐字稿

契约测试听起来很现代，但我希望大家先不要急着学工具，先问一个前置问题：你的系统到底有几个独立部署的服务？

如果前端、后端、通知服务各自独立部署，各自可能单独发布，那么接口变化就可能影响别人，这时契约测试有价值。比如后端把 `status` 字段改成 `state`，前端还按旧字段读取，系统就会出问题。

但如果你们组只是一个 Spring Boot 单体应用，或者后端所有模块都在同一个进程里，强行引入 Pact 可能只是增加复杂度。5 人、6 周的课程项目，很多时候模块化单体加接口测试更合适。

这页真正要训练的不是 Pact，而是技术判断：先进技术不等于适合。先判断部署形态，再决定测试方法。

互动/板书提示

让学生用 20 秒判断本组系统有几个独立部署进程。可能答案：一个后端加一个前端。点评：前后端独立部署可考虑契约测试；后端内部模块通常不需要 Pact。

转场句：明确适用条件后，再看契约测试到底是什么。

【PPT 26】契约测试：消费方与提供方共同验证接口约定

讲授方式：正常讲｜建议时长：3 分钟

本页目标：定义契约测试，说明消费方、契约仓库和提供方各自承担的验证责任。

现场逐字稿

契约测试解决的是独立部署服务之间“接口各自改坏而不自知”的问题。

可以先想一个冲突场景：前端今天没改代码，突然页面坏了；前端说“我没动”，后端说“我接口也能跑”。最后一查，是后端把返回字段从 `status` 改成了 `state`，接口本身返回 200，但前端读不到原字段。

契约测试的做法是：消费方先声明自己期望什么接口，提供方再在自己的 CI 里验证当前实现是否满足这个期望。它不是等完整联调时才发现，而是在双方各自发布前把接口不兼容暴露出来。

可以类比外卖菜单。用户端看到的菜单、价格、规格，必须和商家实际出餐能力一致。如果商家偷偷改了规格字段，用户端还按旧菜单下单，就会出错。契约测试就是双方提前核对菜单。

互动/板书提示

问：契约测试主要验证功能逻辑，还是接口约定？预期答案：接口约定。补充：业务逻辑仍然需要单元测试和集成测试。

转场句：具体工具里，Pact 是常见方案，我们看它的工作流。

【PPT 27】Pact 工作流：接口变化在双方 CI 中暴露

讲授方式：正常讲｜建议时长：4 分钟

本页目标：讲清 Pact 四步流程和它相对端到端测试的提前发现价值。

现场逐字稿

Pact 工作流可以记成四步。

第一，消费方定义契约。比如前端测试写明：请求 `/api/devices?status=available`，期望返回 200，响应体里有 devices 数组，每个设备有 id、name、status。

第二，契约发布到 Pact Broker。它相当于共享契约仓库。第三，提供方在 CI 中拉取契约，用自己的后端 API 去验证。第四，如果字段、状态码或结构不匹配，构建失败。

它的价值是把接口不兼容提前暴露。传统端到端测试要等前后端都部署到测试环境才发现问题；契约测试可以在前端 CI 和后端 CI 各自运行时就发现。

所以 Pact 不是为了显得技术高级，而是为了降低跨团队、跨服务协作中的接口不确定性。没有这个协作问题，就不必硬上这个工具。

互动/板书提示

问：后端把 status 改成 state，Pact 会在哪一步发现？预期答案：提供方验证契约时。点评：这就是把联调问题提前到 CI。

转场句：但不是所有系统都要用 Pact。回到课程项目，我们多数情况下可以用接口测试替代。

【PPT 28】模块化单体：用接口测试替代契约测试

讲授方式：正常讲/快速带过｜建议时长：3 分钟

本页目标：说明模块化单体中接口测试比契约测试更合适，并强调技术选择要匹配复杂度。

现场逐字稿

如果系统是模块化单体，接口测试通常更合适。

模块化单体内部模块在同一代码库、同一进程里，接口变化可以通过集成测试和代码评审同步发现。此时引入 Pact Broker、契约发布、提供方验证，可能比问题本身更复杂。

更实际的做法是启动 Spring 容器，用内存数据库或测试容器，调用 `/api/devices?status=available`，验证 HTTP 状态码、响应结构和核心业务逻辑。这样既检查对外接口，又不引入超出团队规模的工具链。

这正好回到体系结构课的判断：不是技术越多越专业，而是复杂度和收益要匹配。课程项目尤其要避免把大厂微服务工具链不加判断地搬进小团队首版。

互动/板书提示

让学生判断：学院内部版预约系统，5 人 6 周，首版是否必须上 Pact？预期答案：通常不必。点评：接口测试足够，除非确有独立部署和频繁接口变更。

转场句：接下来进入安全测试。安全不是附加项，而是高损失质量属性。

【PPT 29】安全是高损失质量属性

讲授方式：正常讲/快速带过｜建议时长：3 分钟

本页目标：把安全从漏洞科普提升到架构质量属性，强调安全测试要对应架构层。

现场逐字稿

安全是高损失质量属性。功能 bug 可能让一次预约失败，安全漏洞可能让全部学生信息泄露，甚至影响学校账号体 系和科研数据。 很多团队把安全当成最后扫描一下漏洞。但越权、注入、认证绕过这些问题，根往往在架构层。权限在哪里拦截 ，认证在哪里校验，数据访问层是否允许拼 SQL，敏感数据是否加密，这些都是设计决策。 所以本讲不把安全讲成漏洞百科，而是讲“安全问题对应哪一层架构，以及在哪一层测试验证”。只有这样，修 复才不是到处打补丁。 安全之所以要放在体系结构课里讲，是因为安全防线的位置决定了修复成本。一个统一权限层漏了规则，可以集中修；如果每个接口各写各的权限判断，后期修复就像到处补洞，既漏得多，也难验证。

互动/板书提示

问：学生访问管理员接口返回 200，这是测试没测出来，还是架构没有权限层？预期答案：两者都有 ，但根在权限层。

转场句：先看最常见的安全问题：越权访问。

【PPT 30】越权访问：权限层必须集中处理

讲授方式：互动停留/详细讲｜建议时长：5 分钟

本页目标：通过越权访问案例讲清统一权限层、集成测试和 403 断言。

现场逐字稿

越权访问是学生项目里最容易出现、也最危险的问题之一。

场景很简单：普通学生用自己的 Token 请求 `/api/admin/reservations`，结果返回了全部预约记录。不要以为“页面上没有管理员入口”就安全。只要接口存在，别人就可以直接请求。

正确的架构做法是集中权限拦截，比如 Security Filter、AOP 切面、`@PreAuthorize` 注解，而不是依赖每个 Controller 开发者自觉写角色判断。靠自觉一定会漏。

生活里也一样，宿舍楼门禁不能靠每个同学主动问“你是不是本楼的人”，必须有统一门禁设备。软件里的统一权限层就是这个门禁。测试时要用学生 Token 请求管理员接口，期望 HTTP 403，并且响应体不包含任何敏感数据。

互动/板书提示

发问：如果只在前端隐藏管理员菜单，安全吗？预期答案：不安全。点评：前端隐藏不是权限控制，接口层必须拦截。

转场句：第二类安全问题是 SQL 注入，它发生在数据访问层。

【PPT 31】SQL 注入：数据访问层要消除拼接 SQL

讲授方式：正常讲/快速带过｜建议时长：4 分钟

本页目标：讲清 SQL 注入的风险、架构预防方式和测试验证方式。

现场逐字稿

SQL 注入的本质，是把用户输入当成 SQL 语句的一部分执行。 比如设备查询接口接收 device_id，如果代码把输入直接拼到 SQL 里，攻击者传入 `1 OR 1=1`，数据库可能返回不该 返回的数据。这个问题不是靠上线前扫描一下就能彻底解决，根本做法是在数据访问层统一禁止拼接 SQL，使用 ORM、参数化查询或 Repository 规范。 测试时可以构造恶意参数，期望系统返回 400、空结果或统一错误，而不是 200 加全部设备数据。这里要强调：注 入测试不是为了教攻击，而是为了验证数据访问层是否有防线。

互动/板书提示

问：SQL 注入应该主要靠 Controller 里 if 判断过滤，还是靠 Repository 参数化查询？预期答案：参数 化查询。点评：输入校验有用，但架构底线在数据访问层。

转场句：第三类问题是认证绕过，重点在 Token 校验链是否完整。

【PPT 32】认证绕过：Token 校验必须覆盖过期与伪造

讲授方式：正常讲/快速带过｜建议时长：4 分钟

本页目标：说明认证绕过必须验证完整过滤链，而不是只测试登录函数。

现场逐字稿

认证绕过常见于 Token 校验不完整。 比如 JWT 已经过期，但后端只解析了用户 ID，没有检查过期时间；或者只看到了 Token 存在，没有验证签名。这 样用户可能在登出或过期后仍能访问预约接口。 这个问题不能只测登录函数。登录函数能返回 Token，只说明颁发成功；真正的风险在后续每个请求是否都经过完 整 Filter 校验。所以测试要构造过期 Token，请求 `/api/reservations`，期望返回 401。 生活里类似校园卡。办卡成功不等于永远能进楼，门禁每次刷卡都要检查卡是否有效、是否过期、是否有权限。

互动/板书提示

判断题：认证绕过测试为什么通常不是单元测试？预期答案：需要完整 Filter/拦截链。点评：这类测 试要在集成层验证。

转场句：把三类安全问题放到一张表里，就能看到它们分别属于不同架构层。

【PPT 33】安全问题、架构层与测试层对应表

讲授方式：正常讲｜建议时长：4 分钟

本页目标：总结安全问题、架构层和测试层的对应关系，帮助学生做案例分析题。

现场逐字稿

这页表要帮助大家形成一个固定思路：看到安全问题，不要只说“加校验”，而要问它属于哪一层。 越权访问属于权限过滤层或 AOP，测试层通常是集成测试。SQL 注入属于数据访问层，单元和集成测试都可以覆 盖。认证绕过属于 API 网关或 Filter，必须走完整认证链。敏感数据明文属于存储层和传输层，需要代码审查、配 置检查和集成验证。 如果不定位架构层，修复就会变成到处打补丁。今天这个 Controller 补一个 if，明天那个接口补一个判断，最后系 统越来越乱。架构化的安全设计，是把防线放在统一位置，然后用测试证明防线有效。

互动/板书提示

给一个小案例：普通学生能下载其他人的预约附件。让学生判断属于哪一层。可能答案：权限层、 资源访问控制层。点评：要测学生身份访问他人资源是否 403 或 404。

转场句：质量问题被发现以后，还需要进入缺陷生命周期，形成闭环。

【PPT 34】缺陷生命周期：从发现到复盘

讲授方式：正常讲｜建议时长：3 分钟

本页目标：说明缺陷关闭不是终点，补测试和复盘才能形成质量闭环。

现场逐字稿

缺陷生命周期不只是流程图。发现、复现、定级、修复、验证、关闭，这些步骤大家都能理解。但真正容易被忽 略的是“补测试”和“复盘”。 如果并发预约 bug 修好了，但没有把并发测试加入构建门禁，下次重构可能再次引入同类问题。缺陷关闭只是把眼 前问题处理掉，补测试才是把它变成防线，复盘才是把它变成架构改进线索。 企业里很多质

量改进不是来自一次英雄式修复，而是来自缺陷数据积累：哪个模块总出问题，哪类错误总逃逸， 哪个接口总被误用。缺陷是质量数据，不只是 Issue 清单。 复盘时不要只问“谁写错了”，更要问“为什么这类错误能进入主分支”。是没有门禁？是测试层级选错？是架 构边界太模糊？把缺陷追溯到机制层，团队才会持续变好。

互动/板书提示

问：修复 bug 后为什么还要补测试？预期答案：防止回归。点评：补测试就是把一次事故变成长期防 线。

转场句：下一页看一份并发预约缺陷报告，重点看它为什么能指导修复。

【PPT 35】规范缺陷报告：并发预约重复占用

讲授方式：互动停留/详细讲｜建议时长：5 分钟

本页目标：用并发预约缺陷报告示范可复现、可定级、可补回归的写法。

现场逐字稿

好的缺陷报告，目标不是抱怨“系统有 bug”，而是让开发者能复现、能判断优先级、能补回归测试。

这页标题直接说明问题：并发预约同一设备时出现重复占用。重现步骤也清楚：准备设备、固定时段，用 JMeter 20 个线程同时 POST 相同 device_id 和 timeslot。期望结果是 1 个 201，19 个 409，数据库只有 1 条有效记录。实际结果是 3 到 5 个请求都成功，数据库出现多条有效记录。

这种报告有三个优点。第一，它可复现，别人照着能跑出来。第二，它可定级，这是 P0，因为破坏核心数据一致性。第三，它能指导修复：可能缺少乐观锁、悲观锁或唯一约束。

修复后还不能结束，还要把这个并发测试加入门禁。否则这次修好了，下次重构仍然可能回来。

互动/板书提示

让学生把“预约有时候会重复”改写成可复现缺陷报告。给 30 秒思考，点 1 名学生说要补哪些字段。预期答案：环境、步骤、请求参数、期望、实际、优先级。

转场句：缺陷报告不是终点，缺陷数据本身还能反映架构问题。

【PPT 36】缺陷是质量数据，不只是 Issue 清单

讲授方式：正常讲/快速带过｜建议时长：3 分钟

本页目标：解释缺陷数据如何反推架构薄弱点、测试盲区和技术债。

现场逐字稿

如果只是把 Issue 关掉，质量不会自然提升。缺陷积累起来是一种质量数据。 同一服务反复出问题，可能说明这个服务职责太多、边界不清。权限相关 bug 反复出现，往往说明没有统一权限层。 同类缺陷多次逃逸，说明门禁没有覆盖。修复成本越来越高，说明技术债已经影响架构演进。 所以质量保障不是“测试部门把 bug 找出来”这么简单。更成熟的团队会看缺陷趋势：哪些模块缺陷密度高，哪些 缺陷反复出现，哪些修复导致二次 bug。这些数据会反过来推动重构、补门禁、改架构。 在企业里，质量负责人常会看缺陷趋势图和模块缺陷密度。这个数据不是为了追责，而是为了判断下一步应该补 测试、重构模块，还是调整评审重点。学生项目虽然规模小，也可以从最难测、最常出错的地方开始做这个分析。

互动/板书提示

问：如果你们组每次改预约模块都会影响通知模块，说明什么？可能答案：耦合高、边界不清。点 评：缺陷数据可以暴露架构耦合。

转场句：最后讲一个现实问题：AI 能不能帮我们做测试。

【PPT 37】AI 能辅助测试什么

讲授方式：正常讲/快速带过｜建议时长：3 分钟

本页目标：客观说明 AI 在测试中的可用范围，避免学生完全不用或过度依赖。

现场逐字稿

AI 在测试中确实有用。它擅长做测试素材生成和变体枚举。 比如你告诉 AI：预约时长字段要求 1 到 4 小时整数。它很快能帮你列边界值：0、1、4、5、负数、小数、null、超 长字符串。你让它生成 MockDeviceRepository，设备状态返

回 OCCUPIED，它也能很快写出骨架。你把错误栈贴给 它，它能帮助解释可能原因。 但前提是你要给它正确上下文。 AI 不知道你们系统最重要的风险是什么，也不知道你们的架构层级。它像一个执 行力很强的助教，可以帮忙整理题目、生成草稿，但不能替老师决定考试重点。 例如你可以这样问 AI：“请根据预约时长 1 到 4 小时整数这个规则，列出边界值测试用例。”这个问题很适合 AI ，因为规则明确、输出格式清晰。但如果你问“我的系统应该怎么测试”，AI 给出的答案往往泛泛而谈，因为它 不了解你的架构风险。

互动/板书提示

问：AI 适合生成测试骨架吗？适合决定并发测试层级吗？预期答案：前者适合，后者不适合。点评 ：生成和判断要分开。

转场句：下一页专门讲 AI 不能替代什么。

【PPT 38】AI 不能替你决定测试策略

讲授方式：正常讲/快速带过｜建议时长：3 分钟

本页目标：强调 AI 不能替代测试策略选择，测试数量多不等于风险覆盖正确。

现场逐字稿

AI 不能替你决定测试策略。 一个典型误区是：让 AI 生成 100 个单元测试，全部通过，于是觉得质量很好。问题在于，如果核心风险是数据库 并发冲突，这 100 个 Mock 环境里的单元测试都不能证明真实数据库行为正确。测试数量很多，但层级选错，关键 风险仍然逃逸。 这就像考试复习。你刷了 100 道选择题，但考试重点是综合设计题，刷题数量再多也不等于复习到位。软件测试同 样如此：先识别风险，再选择测试层级，最后才让 AI 生成骨架。顺序不能反。 所以评价测试质量不能只看数量，也不能只看覆盖率。覆盖率很高但覆盖的是低风险路径，仍然可能上线出事故。 测试策略的核心是风险优先：最可能出问题、出问题损失最大的地方，要优先被正确层级的测试覆盖。

互动/板书提示

案例提问：AI 生成的单元测试全绿，为什么并发 bug 还会上线？预期答案：单元测试没有覆盖真实 数据库并发。点评：策略错误不是数量能弥补的。

转场句：所以 AI 辅助测试要按正确工作流使用。

【PPT 39】AI 辅助测试的正确工作流

讲授方式：正常讲/快速带过｜建议时长：3 分钟

本页目标：给出可执行的人机协作流程：人做策略，AI 做低风险生成，人再验证。

现场逐字稿

正确流程可以分三段。 第一段，人来做策略判断。你要先识别风险：这个功能最容易出问题的场景是什么？然后决定测试层级：单元、 集成还是端到端。比如并发预约，人必须判断它需要真实数据库，所以是集成测试。 第二段，AI 做生成。你可以让 AI 生成 JMeter 脚本草稿、pytest 测试骨架、Mock 类、边界值列表。第三段，人再验 证。你要检查断言是否符合业务规则，测试失败时是否真的能暴露问题，测试通过时是否说明风险被覆盖。 一句话：AI 负责加速，教师和工程师负责判断。 这套流程也适合课程项目。你们可以先在组内列出三条关键风险，再决定层级，然后让 AI 帮忙生成测试用例草稿。 最后由组长或负责质量的同学检查：断言是否对，环境是否对，是否真的覆盖了风险。

互动/板书提示

让学生记一句话：你做策略，AI 做骨架，你做确认。可让全班一起复述一次。

转场句：讲完方法，接下来进入课堂演练，把第 4 讲的质量属性场景补成测试方案。

【PPT 40】课堂演练：为质量属性场景补“测试方案”列

讲授方式：互动停留/详细讲｜建议时长：6 分钟说明，后续小组填写 15-20 分钟

本页目标：清楚布置课堂演练，让学生知道输入、动作、输出和检查标准。

现场逐字稿

现在进入课堂演练。请各组打开第 4 讲完成的质量属性场景表，也就是包含刺激源、环境、响应、可量化指标的那张表。如果当时没有形成完整表格，现在至少列出你们组最重要的 3 个质量属性场景。

今天要做的动作很明确：在右侧增加一列“测试方案”。每一行都写三件事：测试层级、测试方式、通过标准。

我会把两条检查标准写在黑板上。第一，测试层级选对了吗？并发数据库行为不能写单元测试，纯业务规则不要上端到端。第二，通过标准可量化吗？“响应时间可接受”“不出现明显错误”都不合格；“P95 ≤ 2000ms，错误率 ≤ 0.1%”才可以判断。

接下来各组先做 15 到 20 分钟，优先填并发一致性、通知降级、越权访问这三类场景。

互动/板书提示

操作话术：请各组现在打开场景表，先给 1 分钟准备。随后明确计时 15-20 分钟。教师巡视时只问两句：你们这一行为什么选这个层级？这个通过标准能不能被别人复现？

转场句：在大家动手之前，我先给一个完整示例。

【PPT 41】演练示例：50 并发预约同一设备

讲授方式：互动停留/详细讲｜建议时长：5 分钟

本页目标：用并发预约示例展示测试层级、测试方式和通过标准的合格写法。

现场逐字稿

动手前先看一个完整示例：50 个用户同时预约同一设备。

测试层级写集成测试，因为关键风险在数据库并发控制，需要真实数据库或测试容器，但不需要浏览器。测试方式写清楚：JMeter 50 并发线程，POST `/api/reservations`，相同 device_id 和 timeslot。

通过标准也必须写清楚：201 响应等于 1 条，409 响应等于 49 条，数据库中该时段有效预约记录等于 1 条。这个标准的好处是任何人都能判断，不需要主观解释。

如果有同学写“系统能正确处理并发”，这不合格。什么叫正确？必须落到响应码、数据库记录和错误率。工程语言最重要的特征就是可复现、可判断。

互动/板书提示

让学生说出这个测试为什么不是端到端。预期答案：不需要前端浏览器。点评：端到端不是“真实”的同义词，真实数据库加 API 压测已经覆盖关键风险。

转场句：示例看完后，大家开始填写；填写后还要互审，互审标准看下一页。

【PPT 42】互审标准：层级选对了吗？标准可量化吗？

讲授方式：互动停留/详细讲｜建议时长：6 分钟说明，互审 5-10 分钟

本页目标：说明互审规则，让学生能从层级合理性和标准可量化性两个角度评价他组方案。

现场逐字稿

互审时不要把重点放在排版好不好看，也不要纠结工具名字写得多不多。只看两件事。

第一，层级是否合理。纯逻辑规则不要放端到端；并发数据库行为不要放单元测试；权限过滤链通常要集成测试。第二，标准是否可量化。避免“可接受”“基本正常”“没有明显错误”这些词。要写成 P95 小于多少、错误率小于多少、重复预约数等于多少、HTTP 状态码是多少。

互审时，每组在邻组表格上标出至少两处可以改进的地方。点评要具体，比如“这行并发场景不适合单元测试，因为没有真实数据库事务”，或者“这个通过标准缺少响应时间阈值”。

互审不是挑刺，而是训练工程判断。你能指出邻组测试层级选错，说明你真正理解了测试策略。

互动/板书提示

操作话术：现在交换给相邻组，5-10 分钟。每组至少写 2 条修改建议。教师巡视时挑 1-2 个典型问题，最后全班点评。

转场句：演练结果会直接进入 M4 的第一份文档：测试计划 V1。

【PPT 43】M4 产出一：测试计划 V1

讲授方式：正常讲/快速带过｜建议时长：3 分钟

本页目标：明确测试计划 V1 的结构和质量要求，避免学生只罗列工具。

现场逐字稿

测试计划 V1 不需要写成长篇论文，但必须有实质内容。 第一，测试范围。说明本迭代覆盖哪些核心功能，不覆盖什么，以及为什么不覆盖。第二，测试层级分配。哪些 模块写单元测试，哪些协作场景写集成测试，哪几条链路保留端到端测试。第三，测试环境。你们用本地环境、 内存数据库、Docker Compose，还是真实服务？这些要写清楚。 第四，关键风险。并发一致性、通知降级、越权访问，至少要选出你们系统最重要的 3 个风险，写出测试方案。第 五，AI 辅助说明。哪些测试骨架可以用 AI 生成，哪些策略决策必须由团队自己判断。 测试计划里还要写“不测什么”。这点很重要。真实项目资源有限，不可能什么都测到最高强度。你们可以说明 某些低风险管理页面本次只做手工检查，某些高风险并发和权限场景必须自动化验证。这样计划才有取舍。

互动/板书提示

提醒学生：测试计划不看工具堆得多不多，而看风险是否被覆盖。可问：如果计划里只有“使用 JUnit、Postman、JMeter”，够不够？预期答案：不够。

转场句：M4 第二份文档是质量门禁清单，它决定什么条件下才能进入下一阶段。

【PPT 44】M4 产出二：质量门禁清单

讲授方式：正常讲/快速带过｜建议时长：3 分钟

本页目标：明确质量门禁清单的三层结构和可量化填写要求。

现场逐字稿

质量门禁清单要能被评审老师逐项勾选。 代码层可以写：单元测试通过率 100%，新增代码无 Blocker 级静态分析告警，代码覆盖率达到你们自己设定的目标。 集成层可以写：核心业务流程集成测试全部通过，关键并发场景通过，权限测试通过。发布层可以写：P0 缺陷为 0，越权访问测试通过，P95 响应时间达标，回滚方案准备完成。 这里有一个要求：不要照抄我的数字。并发数、覆盖率、响应时间阈值，要根据你们系统规模填写。小项目可以 目标合理一点，但必须有解释。下一讲会补发布运维方案 V1，和今天的测试计划、质量门禁合并成 M4。

互动/板书提示

让学生在门禁清单里至少填写一个本组自定义数字，例如覆盖率、并发数或响应时间。点评：数字 不是越高越好，而是要与风险和资源匹配。

转场句：最后用七条判断收束本讲，也作为复习和考试锚点。

【PPT 45】本讲七条关键判断

讲授方式：互动停留/详细讲｜建议时长：4 分钟

本页目标：收束全课主线，明确复习锚点和课后行动。

现场逐字稿

最后我们把本讲收成七条判断。不要把它当成七个口号，而是当成后续做 M4 的检查表。

第一，可测试性由架构决定，不是测试工具决定。第二，测试金字塔对应架构层，选错层，测再多也没用。第三，质量门禁是工程约束，不是主观判断。第四，契约测试只适合独立部署服务，模块化单体可以用接口测试替代。第五，安全问题要定位到权限、认证、数据访问等架构层。第六，AI 能生成骨架，但测试策略必须由人决定。第七，测试难写通常暴露了架构耦合点。

课后各组做一件事：回到自己的项目，标出当前最难测试的三个场景，并写清楚难在哪里。是依赖无法替换？状态不可观测？需要真实数据库？还是测试层级选错？这些答案会直接进入你们的 M4 文档。

今天这讲如果只记一句话，就是：质量是被设计出来的，可测试性是架构设计的结果。

互动/板书提示

收束提问：今天这讲如果只记一句话，应该是什么？预期答案：质量是被设计出来的；可测试性由架构决定。教师最后确认这就是本讲主线。

转场句：下一讲我们从“怎样证明系统是对的”，进入“怎样把通过门禁的系统安全发布出去”。